

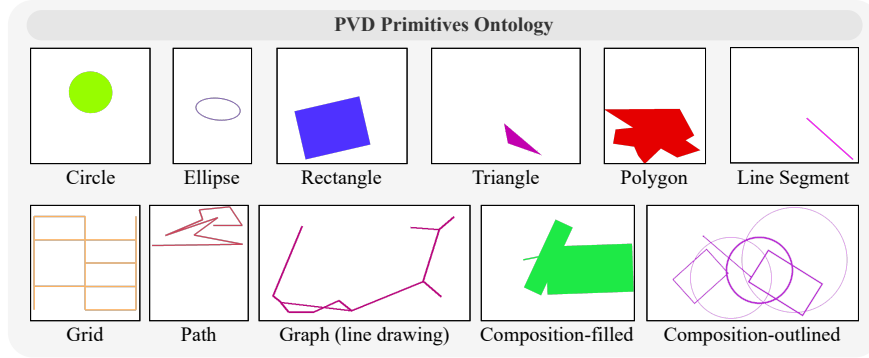


Figure 3: Ontology of the primitives in PVD. Composition of fundamental building blocks for vector graphics, as shown in Figure 4.

2 VDLM Framework

We introduce the VDLM framework, which consists of three main components. First, a rule-based perception module converts images into SVG format, capturing low-level visual details (§ 2.1) that are highly complementary to CLIP-like image features. Second, a trained language model aligns SVGs with intermediate visual descriptions (§ 2.2). Third, an inference-only foundational model reasons about downstream tasks using both visual and textual perception results (§ 2.3). Refer to Figure 2 for an overview of VDLM.

2.1 Precise Visual Perception with SVG Encoding

Prior work (Krojer et al., 2022; Tong et al., 2024) has demonstrated that, although CLIP-based (Radford et al., 2021) vision encoders are effective at capturing high-level visual semantics, they can fall short in preserving fine-grained visual details. We propose extracting an SVG representation that more accurately captures detailed measurements and is highly complementary to the widely used visual features. This can be achieved using rule-based image-to-SVG converters, such as Vtracer VTracer (2024), for which we empirically observe near-perfect reconstruction quality on rasterized vector graphic images (see detailed analysis in Appendix D). SVG describes shapes, lines, and colors using mathematical expressions and paths with precise coordinates.

We conduct a suite of preliminary experiments (§ A) to investigate the potential of using SVG for representing visual inputs. We empirically observe that SVG representation outperforms CLIP-based features on low-level vector graphics reasoning tasks given sufficient task-specific fine-tuning data. However, two key challenges remain (§ A.3): First, off-the-shelf foundation models, such as GPT-4 (OpenAI, 2023a), have limited zero-shot reasoning abilities when dealing with raw SVG code. Second, obtaining task-specific ⟨SVG, question, answer⟩ data for fine-tuning is extremely challenging, which restricts generalization to unseen tasks and domains. We discuss how we address these challenges by learning an intermediate abstraction.

2.2 Bridging Low-Level Visual Perception with High-Level Language Reasoning

Primal Visual Description (PVD). We propose Primal Visual Description, a higher-level abstraction that transforms low-level SVG paths to more structured primitives required for reasoning. PVD is a text-based visual description that consists of a set of primitive geometry objects, e.g., circles and line segments. Each PVD element contains the primitives’ attributes (e.g., color, shape, position, size) with corresponding predicted values (e.g., blue, circle, pixel coordinates of the center, length of the radius). An example of the PVD representation is as follows (See Figure 14 for full definitions):

```
{
  "type": "circle",
  "center": [252, 315],
  "radius": 202,
  "color": [175, 155, 98],
  "style": "filled shape"
}
```

Unlike raw SVG code, PVD can be *directly reasoned about* by strong off-the-shelf foundation models to generalize across various downstream tasks. Moreover, PVD is sufficient to serve as a unified visual description across different types of vector graphics, as complex concepts can be composed of multiple primitive shapes. For example, a “cross” can be composed of two “rectangles.” As shown in Figure 3, the ontology of the Primal Visual Description contains 9 canonical primitive shape types that can be composed to cover various vector graphics in the wild. The primitive shapes include circles, ellipses, rectangles, triangles, polygons, line segments, grids, paths, and graphs. A path in PVD is defined as a non-intersecting polyline. Graphs and grids are defined as a set of vertices connected by a set of edges. As an initial step to build a visually descriptive intermediate representation, we focus on demonstrating proof of concept; extension to a more comprehensive ontology will be left for future work.

Learning SVG-to-PVD alignment with a language model. We then train a language model to generate PVD outputs from SVG inputs. The input is a single SVG path depicting a visual concept, and the output is the predicted one or more primitives in the defined PVD ontology. During inference, given an arbitrary raster image, we first convert it into a raw SVG file, which may contain a large number of SVG paths, including noise and speckles. To denoise the raw SVG file and extract salient shapes, we propose an incremental decomposition algorithm. Specifically, we incrementally include SVG paths while checking the difference between the partially rendered image of currently chosen paths and the fully rendered image of the original raw SVG file. We compute the summation of the absolute pixel-by-pixel difference between the two images and set an empirical threshold. If the difference after adding a new path is below this threshold, i.e., if the added path does not bring much additional visual information to the scene, we will skip that path. For the ordering of the path selection, we follow the default ordering from VTracer (2024) that heuristically places the paths with a larger area at the front. The paths that come afterward will be stacked on top of previous paths during rendering. Upon obtaining the decomposed single SVG paths, we first generate their PVD representation individually. We then aggregate the individual PVD predictions into a holistic perception of the entire image using this JSON template: `["object_0": <PVD output for path 0>, "object_1": <PVD output for path 1>, ...]`.

Importantly, since PVD is **task-agnostic**, the data for training the SVG-to-PVD model can be procedurally generated without human annotation. We develop a data generator leveraging PIL.ImageDraw and VTracer (2024), which creates a large-scale (SVG, PVD) paired dataset containing randomly generated primitives. In some real-world tasks, such as geometry problems, multiple primitive shapes with the same color can overlap. When converted to SVG, these shapes tend to be parsed into one merged SVG path. To enable the SVG-to-PVD model to learn to decode individual primitives from such compositional concepts, we additionally generate data instances with randomly overlapped shapes. The target PVD representation, in this context, is a list of primitive PVD JSON objects. We ensure that each generated image contains only one unicolor object, single or composed, so that the converted SVG contains a single SVG path. This facilitates a language model in effectively learning the alignment between SVG and PVD.

To improve the robustness to unseen inference images, we randomize the image sizes, the positions and rotations of the shapes, as well as the styles of the shapes (filled or outlined). We additionally use two data augmentation methods, Gaussian Blur and Pixel Noise, to add variance to the training SVG paths. Our final dataset contains 160K (SVG, PVD) pairs. More details can be found in Appendix C.

We fine-tune a pretrained Mistral-7b (Jiang et al., 2023) model on the synthesized PVD 160K dataset to perform SVG-to-PVD generation. We conduct full-parameter fine-tuning for 3 epochs with a learning rate of $1e-5$. The training objective is a standard Language Modeling loss on the generated PVD tokens as follows:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log P(\mathbf{d}_i | \mathbf{s}, \mathbf{d}_{0:i-1}) \quad (1)$$

where $\mathbf{s}$ and $\mathbf{d}$ refer to the input SVG tokens and the generated PVD tokens respectively. We use the Megatron-LLM (Cano et al., 2023) library for efficient LLM fine-tuning and the entire training process can be done in 16 hours on 4 NVIDIA A100-40GB GPUs.

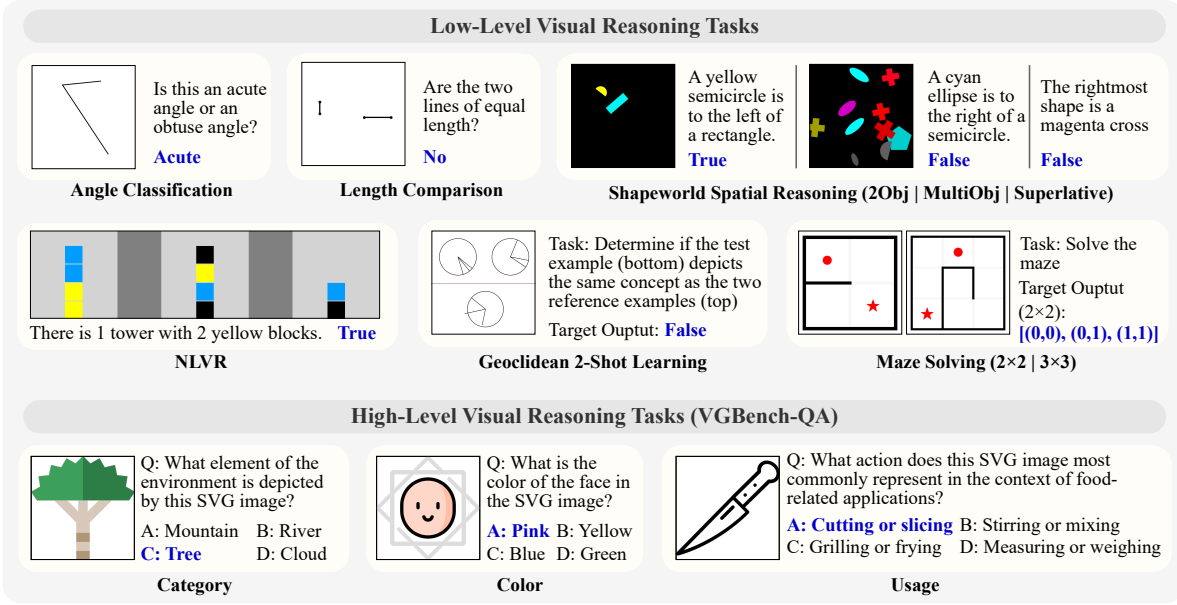


Figure 4: Our full evaluation benchmark with a focus on **low-level visual reasoning** about vector graphics (detailed in 3.1). We additionally include high-level reasoning tasks with rendered SVG images from VGBench-QA (Zou et al., 2024). All tasks are evaluated in a zero-shot setting.

2.3 Enhancing Low-Level Visual Reasoning with Primal Visual Descriptions

PVD provides precise visual details that are highly complementary to the semantic-centric visual features from pretrained visual encoders, such as CLIP. Its textual nature further facilitates direct integration into an inference-only LLM or LMM reasoner.

We explore two variants of VDLM: **VDLM-mm** and **VDLM-txt**, based on the type of reasoner applied. By default, VDLM utilizes a multimodal LMM as the reasoner, referred to as VDLM-mm, which takes both the original image and the PVD perception as input. To examine the efficacy of using PVD to represent visual information, we also consider VDLM-txt, which uses a text-only LLM as the reasoner. A detailed execution trace of the VDLM functions is illustrated in Figure 2. We observe that a strong reasoner, such as GPT-4 (OpenAI, 2023a), without any fine-tuning, can effectively perform various types of task-specific reasoning based on the PVD representation. This includes identifying higher-level concepts, computing measurements, examining spatial relations, and performing multi-step reasoning. The reasoning procedure is also more explainable and transparent compared to the output of existing monolithic LMMs.

3 Experiments

3.1 Tasks

Low-level visual reasoning tasks. Evaluating LMMs in tasks that require precise visual perception about vector graphics is a highly underexplored research area and has limited existing resources. To this end, we construct a new evaluation benchmark that comprises 9 tasks which cover important aspects of low-level visual perception and reasoning, including measurements, spatial relations, counting, logical reasoning, and complex reasoning problems such as maze solving. The description of each task is as follows: (1) **Angle Classification:** Identify whether an angle is acute or obtuse. (2) **Length Comparison:** Determine whether two line segments are of equal length. (3-4) **Shapeworld Spatial Reasoning:** The Shapeworld (Kuhnle & Copestake, 2017) dataset on spatial relations with images containing exactly two objects or multiple objects. (5) **Shapeworld Superlative:** The Shapeworld dataset on superlative statements. (6) **NLVR:** The Natural Language for Visual Reasoning dataset (Suhr et al., 2017) which contains diverse counting, spatial reasoning, and logical reasoning queries. (7) **Geoclidean 2-shot Learning:** A repurposed Geoclidean (Hsu et al.,